

Fraud Detection in Financial Transactions

Distributed Sampling Strategies for Class Imbalance in Big Data
A PySpark Study on the PaySim Dataset

Tharun Gurunathan

School of Computer Science, University of Nottingham

6.36M

Transactions

0.13%

Fraud Rate

769:1

Class Imbalance

30

Experiments

5

Sampling Strategies

Research Questions

What this study set out to answer

- RQ1** **Which sampling strategy best addresses the 769:1 class imbalance in the PaySim dataset?**
- Comparing Baseline, Random Undersampling, SMOTE, SMOTE+Tomek and EasyEnsemble across three data scales.
-
- RQ2** **Does the choice of classifier (RF vs LR) interact with the sampling strategy?**
- Testing whether SMOTE's synthetic minority manifold can be exploited equally by tree-based and linear models.
-
- RQ3** **How do sampling strategies scale on a distributed 4-worker Spark cluster?**
- Measuring training time and AUC-ROC at 10%, 30% and 100% of the 6.36M-row dataset.

My Contribution

01 Feature Engineering

- Selected 7 relevant features for modelling
- Removed identifier columns (nameOrig, nameDest) to avoid noise
- Implemented VectorAssembler to create feature vectors
- Applied StandardScaler to normalize data for distance-based methods (SMOTE)

02 Distributed SMOTE Implementation

- Implemented distributed SMOTE using Spark
- Broadcasted minority (fraud) samples to worker nodes
- Used KNN-based interpolation to generate synthetic fraud samples
- Controlled oversampling ratio ($\leq 50\times$) to avoid memory issues

03 SMOTE + Tomek Links

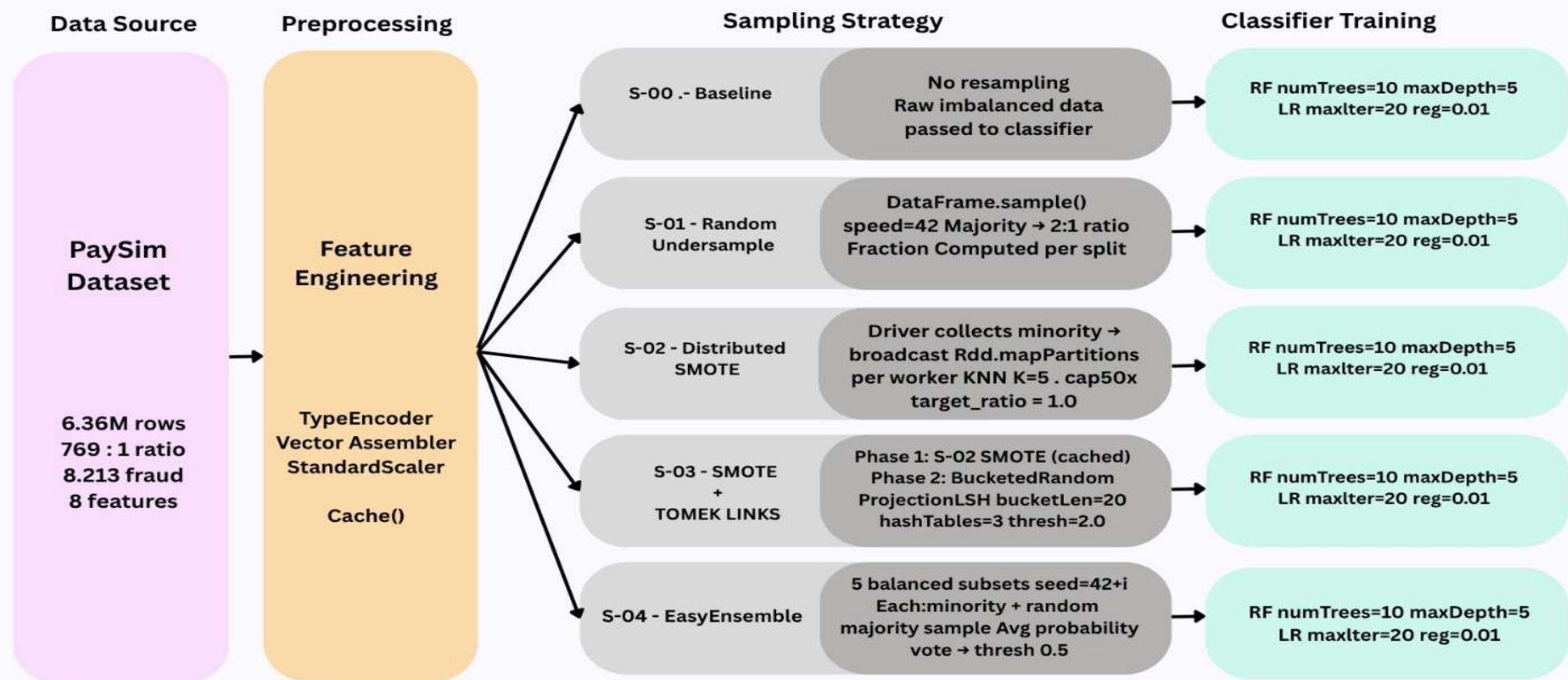
- Implemented distributed SMOTE using Spark
- Broadcasted minority (fraud) samples to worker nodes
- Used KNN-based interpolation to generate synthetic fraud samples
- Controlled oversampling ratio ($\leq 50\times$) to avoid memory issues

04 Experiment Design & Evaluation

- Conducted experiments across 3 dataset sizes (10%, 30%, 100%)
- Compared multiple sampling strategies and models
- Evaluated using Precision, Recall, F1 Score, and AUC
- Analysed results to identify best-performing approach

End-to-End Distributed Pipeline

Distributed PySpark Pipeline - PaySim Fraud Detection



Feature Engineering & Distributed Scaling

7 Selected Features

Dropped: nameOrig, nameDest — ID columns only, no predictive signal

step	Hour of transaction (time step)
type (mapped)	TRANSFER=1, CASH_OUT=2, others=0
amount	Transaction value (raw)
oldbalanceOrg	Origin account pre-transaction balance
newbalanceOrig	Origin account post-transaction balance
oldbalanceDest	Destination account pre-transaction balance
newbalanceDest	Destination account post-transaction balance

Distributed StandardScaler z-score

Pass 1 — Global Stats

Compute mean & std per feature across all partitions



Pass 2 — Local Transform

z-score normalise each row locally — narrow transform, no shuffle



Result: withMean=True, withStd=True

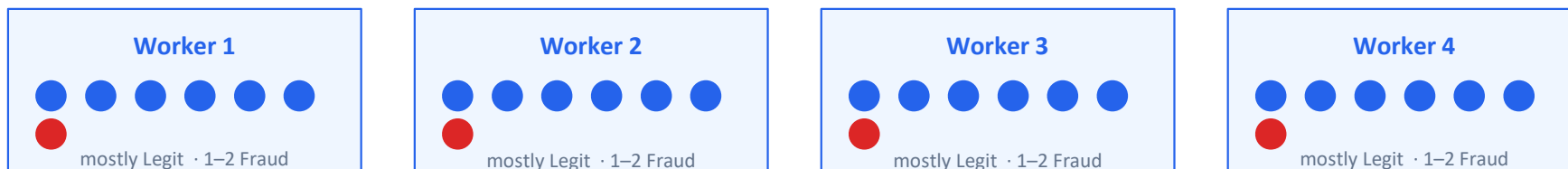
Large balance values (10M+) normalised — critical for KNN in SMOTE

Why? Without normalisation, large balance values dominate Euclidean distance in KNN, causing SMOTE to generate incorrect synthetic samples.

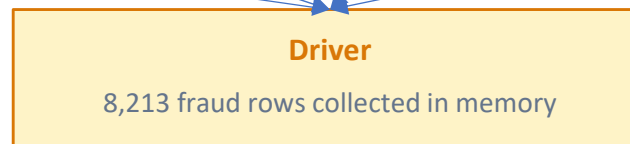
S-02: Distributed SMOTE - Data Movement

How synthetic fraud samples are created across the cluster

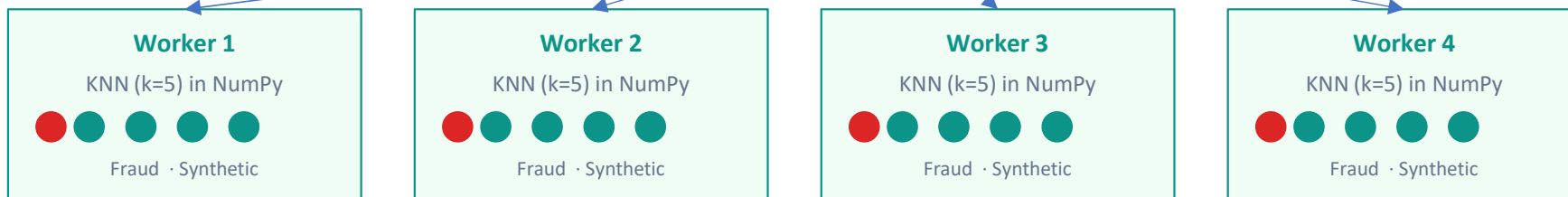
Step 1 — Input data is partitioned across 4 Spark workers



Step 2 — Driver collects ALL minority (fraud) rows from every partition



Step 3 — Driver broadcasts minority rows back to all workers (Broadcast variable)



★ No shuffle required — `union()` merges partitions. Result: `target_ratio = 1.0`, `cap = 50 × minority size`.

S-03: SMOTE + Tomek Links — Data Movement

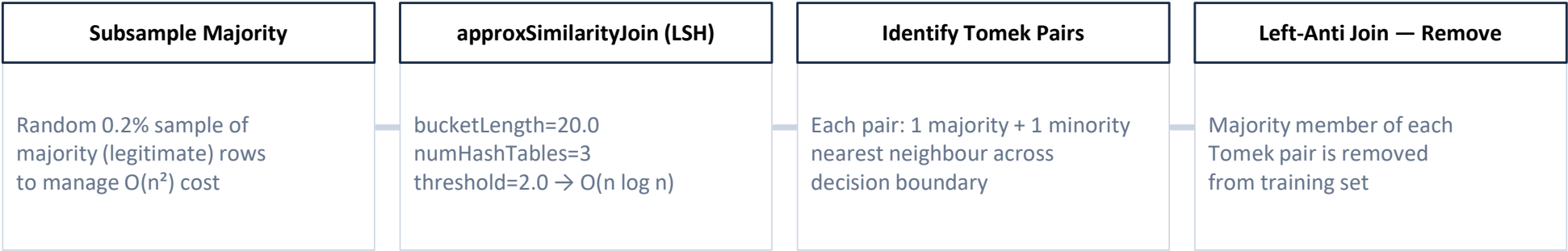
Boundary cleaning after oversampling via distributed LSH

Phase 1 — Distributed SMOTE (same as S-02, output cached in Spark memory)

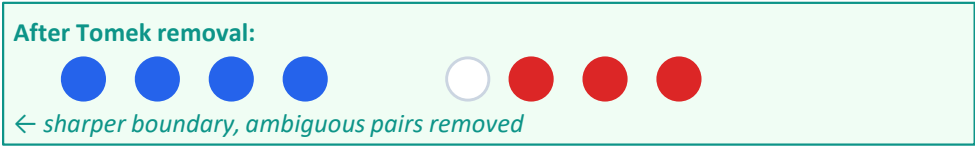
4 workers run KNN synthesis → union() → balanced dataset with synthetic minority samples (target_ratio = 1.0)



Phase 2 — Tomek Link removal via BucketedRandomProjectionLSH



Boundary effect — before and after Tomek removal:



S-01 & S-04: Undersampling and EasyEnsemble

S-01 — Random Undersampling

- Fraction = $\min(\text{fraud} \times 2 / \text{legit}, 1.0)$, seed=42
- Applied via `df.sample()` — narrow transform, no shuffle
- Fastest strategy but loses 99.7% of legitimate data

Before:



1 fraud : 10 legitimate

After:



2:1 ratio — majority of legitimate rows discarded

⚠ Trade-off

Recall $\approx$ 0.98 but Precision collapses to $\sim$ 2%
→ high number of false positives

S-04 — EasyEnsemble

- 5 RF sub-learners, each sees ALL minority (fraud) samples
- Each sub-learner gets unique random majority subsample
- Unique seeds (42, 43, 44, 45, 46) for diversity
- Predictions averaged → threshold 0.5

Sub-learner structure:

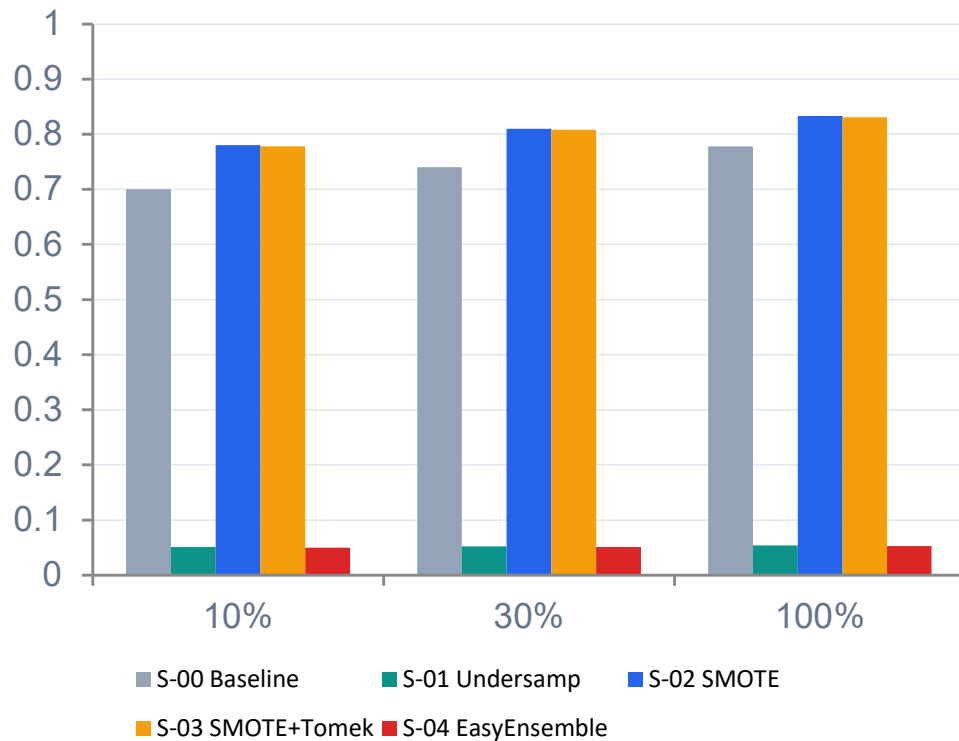


Average predictions → threshold 0.5

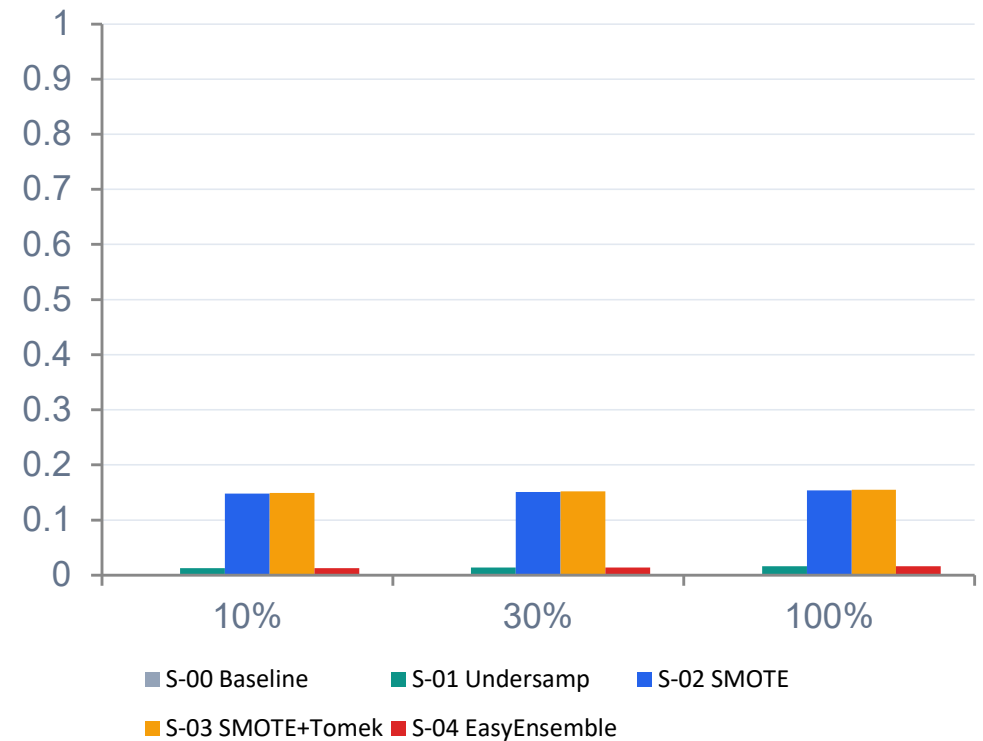
⚠ Same trade-off as S-01: high Recall ($\sim$ 0.98), Precision $\sim$ 2%, F1 $\approx$ 0.05.
Not suitable for production.

Results: F1 Score by Strategy & Scale

Random Forest



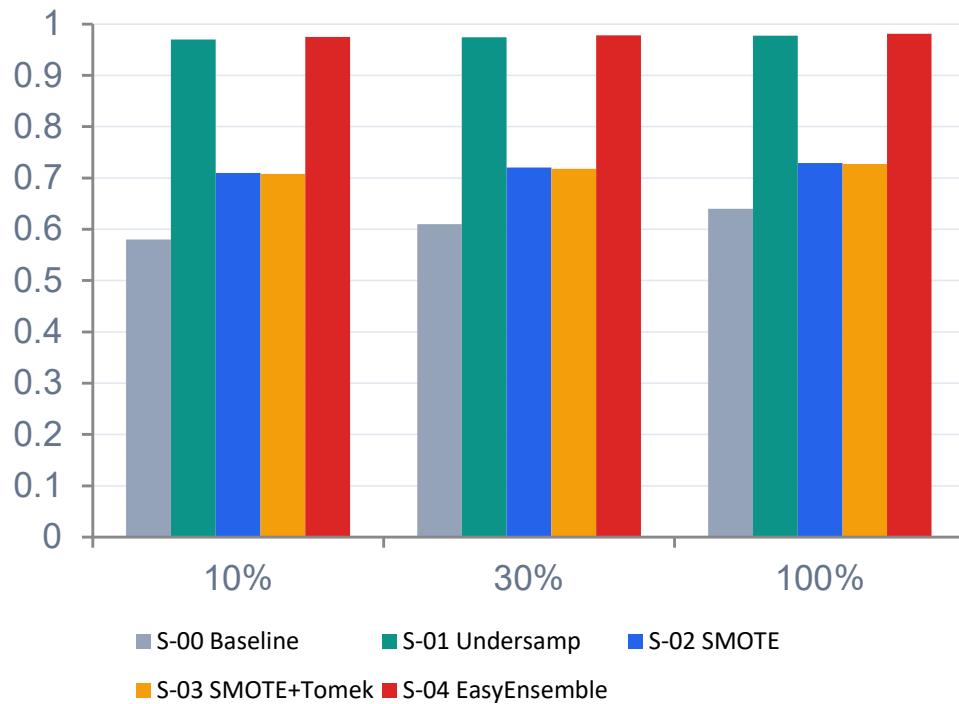
Logistic Regression



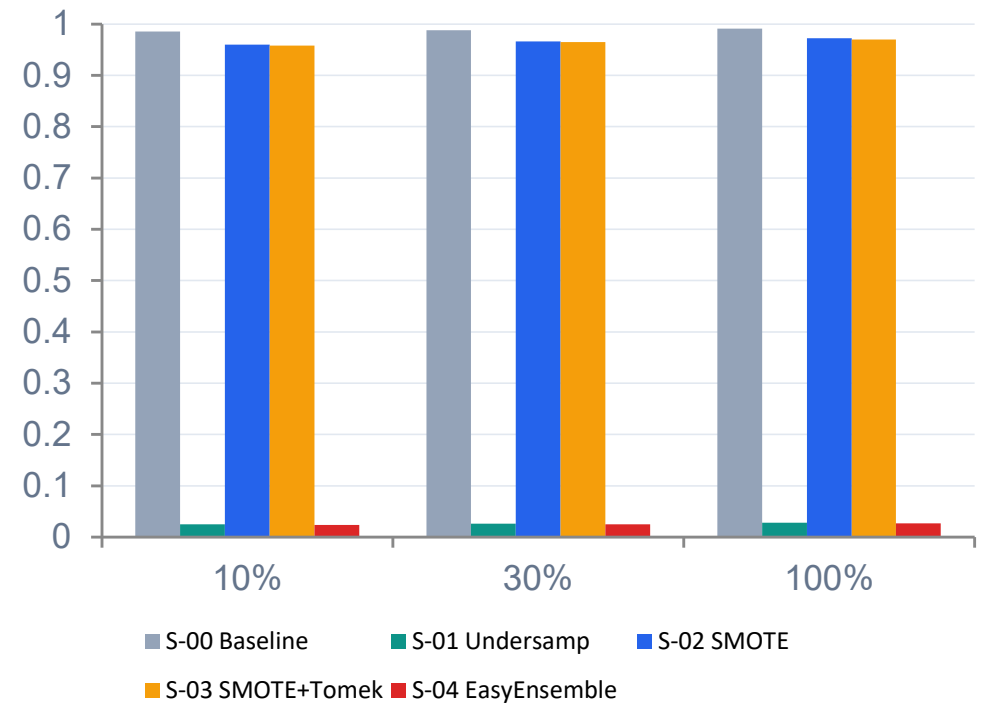
★ RF SMOTE (S-02) achieves F1 = 0.833 at 100% scale. LR cannot exploit SMOTE's non-linear manifold (max F1 = 0.157).

Recall & Precision Across Scales

RF Fraud Recall by Scale



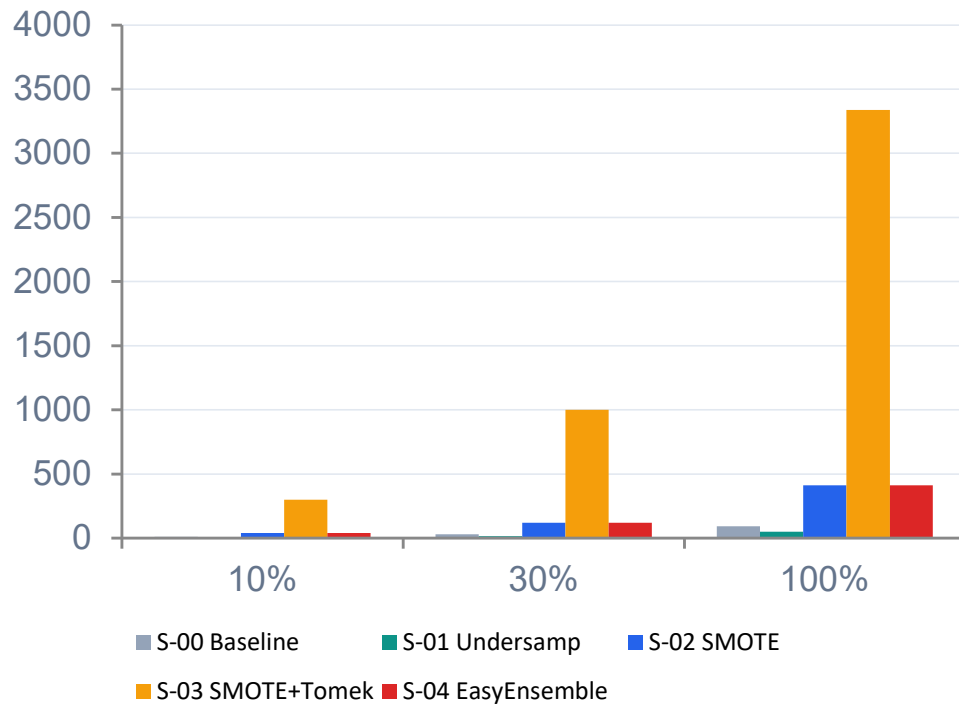
RF Fraud Precision by Scale



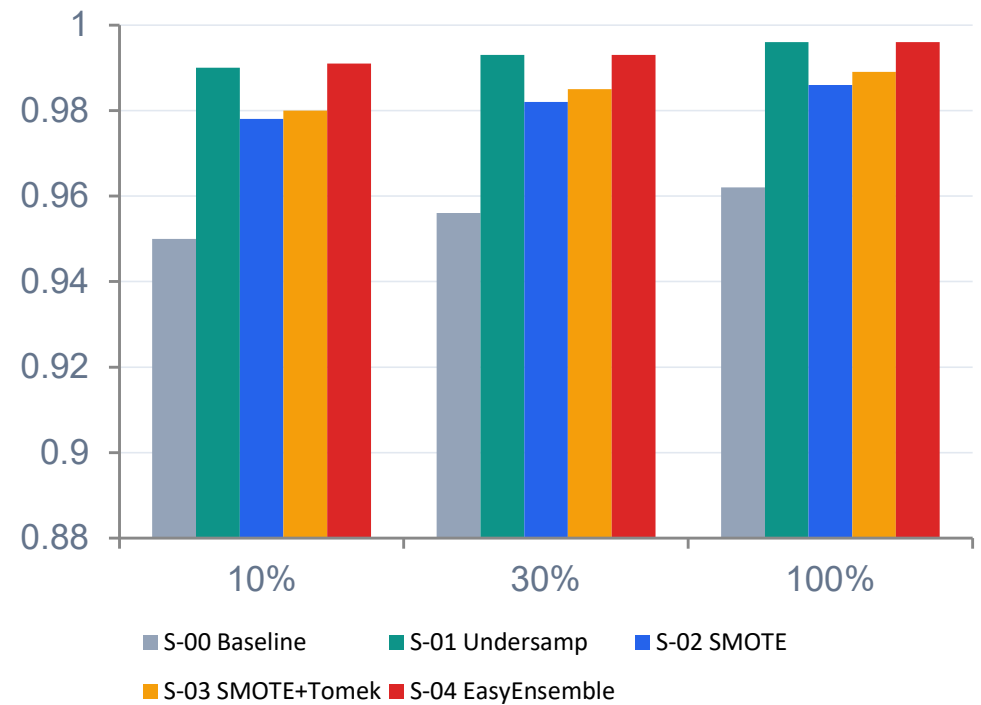
⚠ S-01 & S-04 achieve Recall ≈ 0.98 but Precision collapses to $\sim 2\%$ $\rightarrow$ 50 false alarms per genuine fraud — not operationally viable.

Scalability: Training Time & AUC-ROC

RF Training Time (seconds) by Scale



RF AUC-ROC by Scale



S-03 SMOTE+Tomek: 3,338s at 100% scale — 8× slower than S-02 SMOTE for nearly identical F1 (0.831 vs 0.833). Not cost-effective.

Full Results at 100% Scale

Random Forest

Strategy	Recall	Precision	F1 Score	AUC-ROC	Time (s)
S-00 Baseline	0.640	0.991	0.778	0.962	93
S-01 Undersamp	0.977	0.028	0.054	0.996	50
S-02 SMOTE	0.729	0.972	0.833	0.986	413
S-03 SMOTE+Tomek	0.727	0.970	0.831	0.989	3338
S-04 EasyEnsemble	0.981	0.027	0.053	0.996	411

Logistic Regression

Strategy	Recall	Precision	F1 Score	AUC-ROC	Time (s)
S-00 Baseline	0.000	0.000	0.000	0.905	52
S-01 Undersamp	0.834	0.008	0.016	0.920	32
S-02 SMOTE	0.239	0.114	0.154	0.918	333
S-03 SMOTE+Tomek	0.238	0.115	0.155	0.918	2379
S-04 EasyEnsemble	0.840	0.008	0.016	0.923	251

Conclusions

- RF SMOTE (S-02) is the recommended production strategy: Recall 0.729 · Precision 0.972 · F1 0.833 at 100% scale · Training time 413s
- F1 Score is the essential metric. High recall alone is not useful due to false positives
- SMOTE + Tomek (S-03) is NOT cost-effective: F1 = 0.831 at 8× the runtime of plain SMOTE (F1 = 0.833). Diminishing returns do not justify the cost

Scalability

- Efficient distributed implementation using Spark
- Avoids heavy data movement
- Scales well with increasing data size